

Guía de Flet para Aplicación de Estacionamiento

Controles de UI, fragmentos de código y lógica esencial en Python

1. Resumen de Controles de Flet Recomendados

Para construir el panel de control del estacionamiento, estos son los componentes visuales principales que utilizarás:

Elemento / Control	Clase en Flet	Uso Principal en la App
Lugar / Cochera	<code>ft.Container</code> o <code>ft.Card</code>	Representa visualmente cada espacio (cambia de color verde/rojo).
Grilla de Lugares	<code>ft.GridView</code>	Organiza las cocheras en filas y columnas ordenadas.
Entrada de Texto	<code>ft.TextField</code>	Captura la patente/placa del vehículo.
Selección Tipo	<code>ft Dropdown</code>	Selección de vehículo (Auto, Moto, Camioneta).
Botón Acción	<code>ft.ElevatedButton</code>	Registra el ingreso o procesa la salida.
Ventana Cobro	<code>ft.AlertDialog</code>	Muestra el resumen y total a pagar al liberar el lugar.
Tabla Historial	<code>ft.DataTable</code>	Muestra la lista de registros e ingresos cobrados.

2. Estructura Básica de una Aplicación en Flet

Todo programa en Flet inicia configurando la página principal y la función contenedora.

```
import flet as ft

def main(page: ft.Page):
    page.title = "Sistema de Estacionamiento"
    page.padding = 20
    page.theme_mode = ft.ThemeMode.LIGHT

    # Agregar componentes a la pantalla
    page.add(
        ft.Text("Control de Estacionamiento", size=24, weight=ft.FontWeight.BOLD)
    )

ft.app(target=main)
```

3. Lógica para la Grilla de Cocheras

Creación de una matriz visual de espacios dinámicos que reaccionan al hacer clic sobre ellos.

```

import flet as ft

def crear_cochera(numero, ocupado=False):
    def al_hacer_click(e):
        print(f"Cochera {numero} seleccionada")

    return ft.Container(
        content=ft.Text(f"N° {numero}", color=ft.colors.WHITE, weight=ft.FontWeight.BOLD),
        alignment=ft.alignment.center,
        width=100,
        height=100,
        bgcolor=ft.colors.RED_600 if ocupado else ft.colors.GREEN_600,
        border_radius=8,
        on_click=al_hacer_click
    )

def main(page: ft.Page):
    grid = ft.GridView(expand=True, max_extent=120, child_aspect_ratio=1.0, spacing=10)

    # Crear 12 cocheras de ejemplo
    for i in range(1, 13):
        grid.controls.append(crear_cochera(numero=i, ocupado=(i % 3 == 0)))

    page.add(grid)

ft.app(target=main)

```

4. Formulario de Registro de Vehículos

Ejemplo para tomar los datos de la patente y el tipo de vehículo al presionar un botón.

```

import flet as ft

def main(page: ft.Page):
    txt_patente = ft.TextField(label="Patente / Placa", width=200)
    dd_tipo = ft.Dropdown(
        label="Tipo",
        width=150,
        options=[
            ft.dropdown.Option("Auto"),
            ft.dropdown.Option("Moto"),
            ft.dropdown.Option("Camioneta"),
        ]
    )

    def registrar_ingreso(e):
        if not txt_patente.value:
            txt_patente.error_text = "Ingrese patente"
            page.update()
            return

        print(f"Ingreso: {txt_patente.value} - Tipo: {dd_tipo.value}")
        txt_patente.value = ""
        txt_patente.error_text = None
        page.update()

    btn_registrar = ft.ElevatedButton("Registrar Entrada", on_click=registrar_ingreso)

    page.add(
        ft.Row([txt_patente, dd_tipo, btn_registrar])
    )

ft.app(target=main)

```

5. Lógica de Cálculo de Tarifa (datetime)

Cálculo exacto del costo basado en el tiempo transcurrido entre la entrada y la salida.

```

from datetime import datetime
import math

TARIFA_POR_HORA = 1500 # Valor base por hora

def calcular_monto(hora_entrada: datetime, hora_salida: datetime):
    diferencia = hora_salida - hora_entrada
    segundos_totales = diferencia.total_seconds()

    # Convertir a horas redondeando hacia arriba (mínimo 1 hora)
    horas = math.ceil(segundos_totales / 3600)
    if horas == 0:
        horas = 1

    total = horas * TARIFA_POR_HORA
    return horas, total

# Ejemplo de prueba:
entrada = datetime(2026, 7, 27, 14, 0, 0)
salida = datetime(2026, 7, 27, 16, 15, 0) # 2 hs y 15 min -> Cobra 3 hs

horas_cobradas, total_a_pagar = calcular_monto(entrada, salida)
print(f"Horas: {horas_cobradas} | Total: ${total_a_pagar}")

```

Tip de Organización: Puedes separar tu proyecto en tres archivos: `main.py` (interfaz Flet), `database.py` (guardado de datos) y `logic.py` (cálculo de tiempos y tarifas).